IT 112: Web Systems & Technologies

BSIT-2B 2nd Sem

S.Y. 2025-2026

Final Project Proposal

Phase 6 Documentation for the Quick Reserve :Online Bus & Travel Booking System A Final Project by the <Tech_ Vanguard />(Week 15)

Submitted By:

Nathaniel O. Raynada

Anna Mae P. Broma

Camille T. Bercilla

Mariel O. Odoño

John Edmar L. Bongcodin

Justin James Dasallas

Submitted To:

Dr. Guillermo V. Red, Jr, DIT

May 2026

I. Overview :Update and Delete Functionalities

The QuickReserve System is designed to provide an efficient and user-friendly platform for managing reservations through a complete set of CRUD (Create, Read, Update, Delete) operations. At this stage of development, the system emphasizes the implementation of update and delete functionalities, which are essential for maintaining accurate and reliable reservation data. These features allow both users and administrators to have better control over bookings by enabling them to modify existing entries or remove records that are no longer needed.

The update functionality plays a crucial role in ensuring that reservation details remain accurate and up to date. Using the PUT method, users can edit important information such as customer name, contact details, reservation date and time, and selected services. When the user initiates an edit action, the system displays a modal form with prefilled data retrieved from the database. This approach minimizes input errors and improves user experience by allowing quick and convenient modifications. Once the form is submitted, the updated data is sent to the backend server, processed, and then saved in the MongoDB database, immediately reflecting the changes in the system.

In addition to updating records, the system also supports the deletion of reservation data through the DELETE method. This functionality is important for handling cancelled bookings, duplicate entries, or outdated information. To ensure data safety, the system includes a confirmation prompt before performing the deletion action, preventing accidental removal of records. After confirmation, the selected reservation is permanently deleted from the database, and the user interface is automatically refreshed to display the updated list of reservations. This process ensures that users always see the most current and relevant data.

In the end, the QuickReserve System demonstrates a strong integration between the frontend, backend, and database components. The use of API requests enables smooth communication between the user interface and the server, while MongoDB ensures efficient data storage and retrieval. With the successful implementation of update and delete functionalities, the system is now capable of fully managing reservation data in real time. This enhances the reliability, usability, and overall performance of the application, making it more practical for real-world use and deployment.

II. Screenshots of Edit Modal / Page

The edit functionality allows users to update reservation details in the QuickReserve System. When the edit button is clicked, a modal form appears with prefilled data from the selected record. This makes it easier for users to modify information such as name, schedule, and service without re-entering all data**.**

Figure 1. Admin Dashboard Interface

This screenshot shows the main dashboard of the QuickReserve Admin Panel. The dashboard provides an overview of the system, including the total number of bookings, registered users, active routes, and total revenue. It also displays booking statistics and recent reservations to help the administrator monitor system activities efficiently.

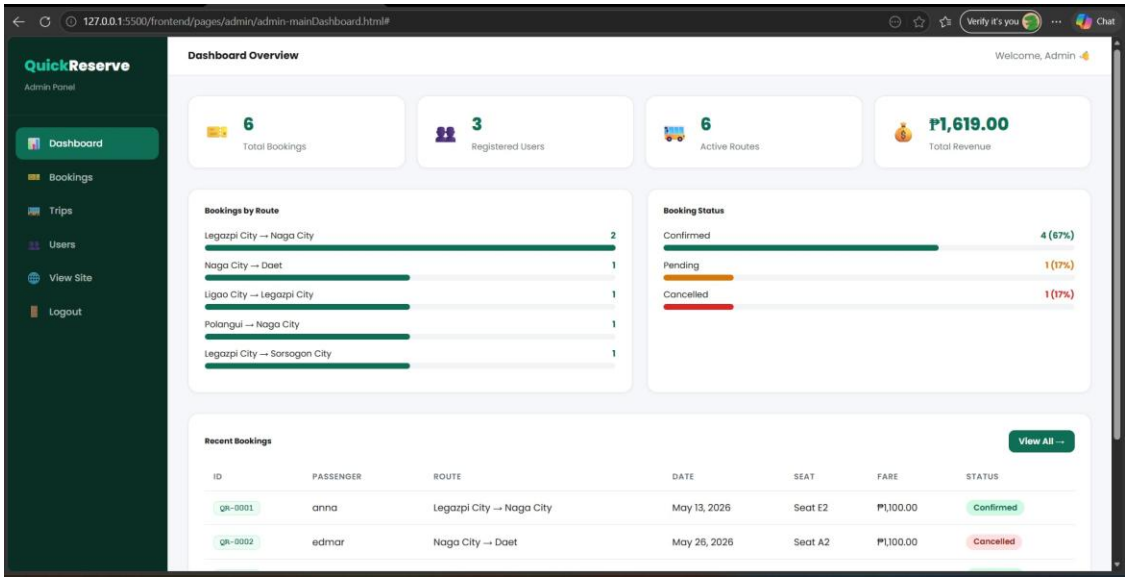


Figure 2. Manage Bookings Page

This screenshot presents the Manage Bookings section of the QuickReserve System. The administrator can view all reservation records, including passenger details, routes, schedules, payment methods, and booking status. The page also contains Edit and Delete buttons that allow administrators to update or remove reservation information easily.

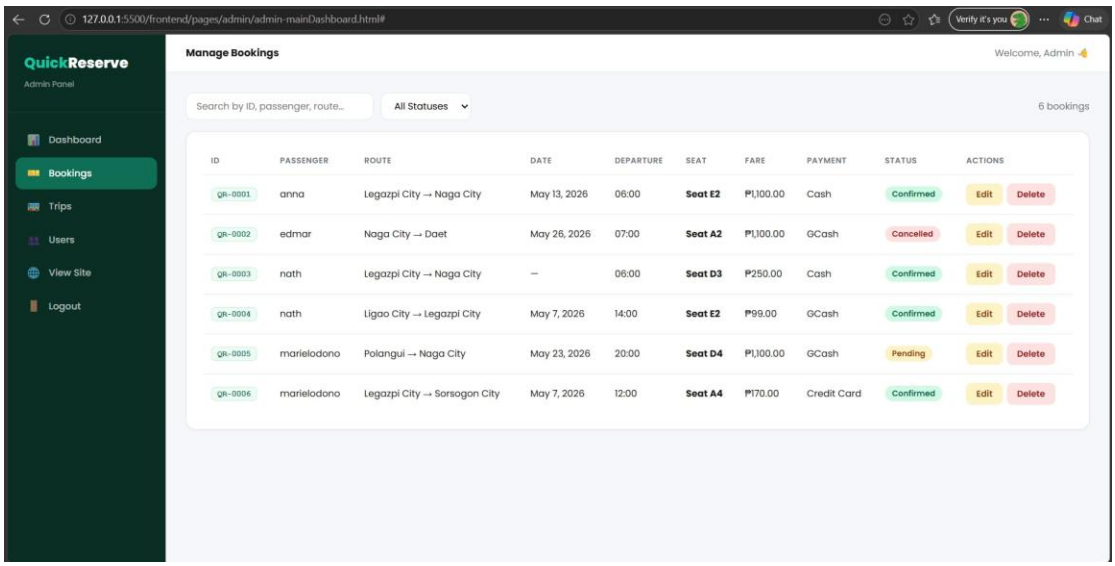


Figure 3. Manage Trips / Routes Page

This screenshot displays the Manage Trips / Routes section of the admin panel. Administrators can manage available travel routes, fare ranges, departure schedules, and seating capacity. The Edit and Delete buttons are provided to modify or remove trip information, while the “Add Trip” button allows administrators to create new routes in the system.

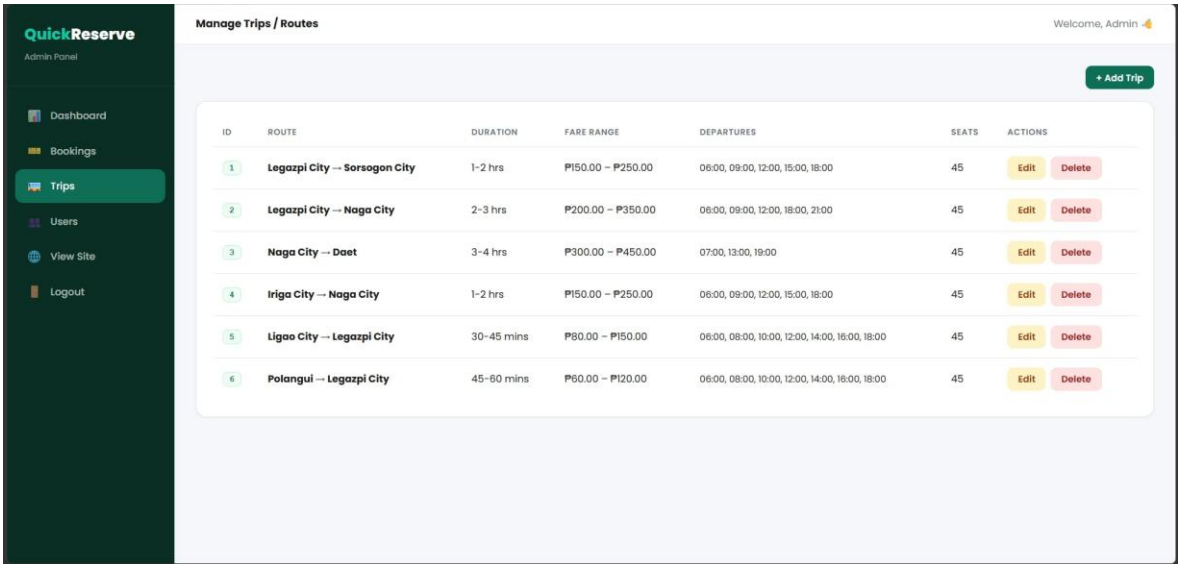
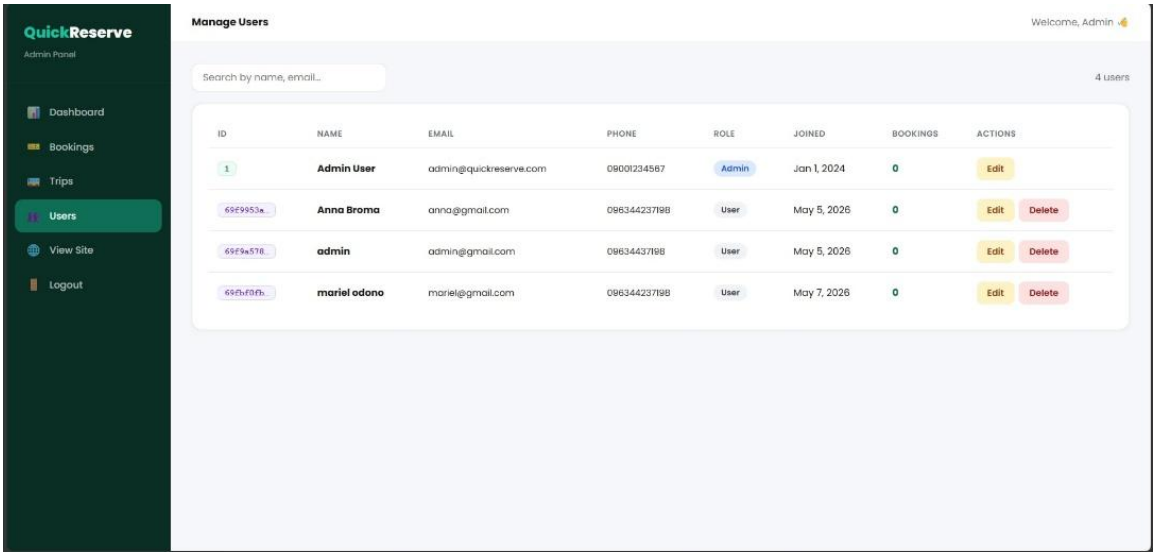


Figure 4. Manage Users Page

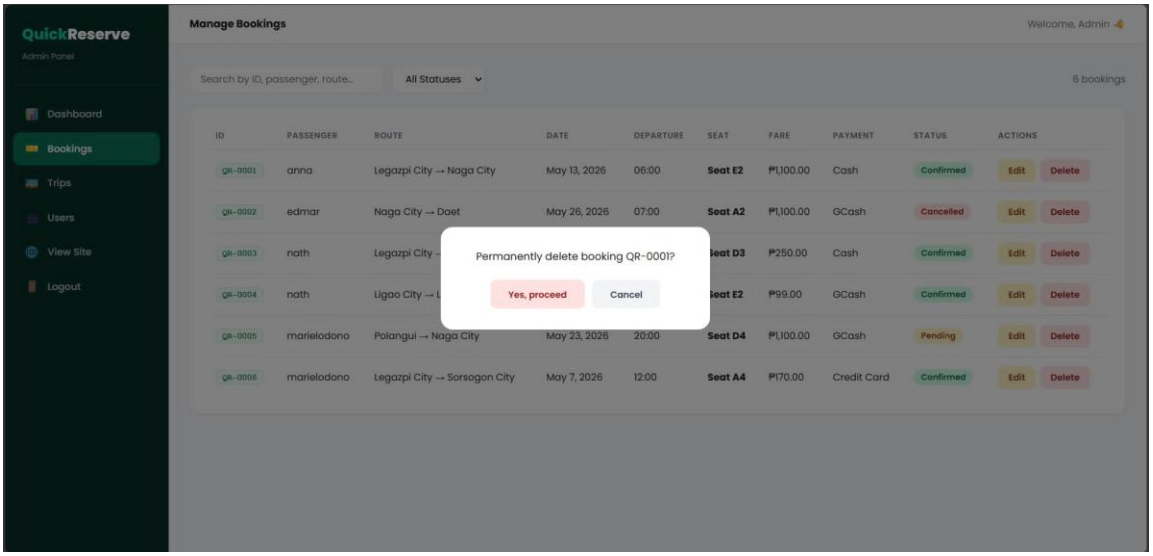
The image shows the **Manage Users** page of the **QuickReserve Admin Panel**. It displays a table of registered users with details such as ID, name, email, phone number, role, joined date, and bookings. Admins can search, edit, or delete user accounts through the dashboard interface.



III. Screenshots of Delete Confirmation and Database Updates

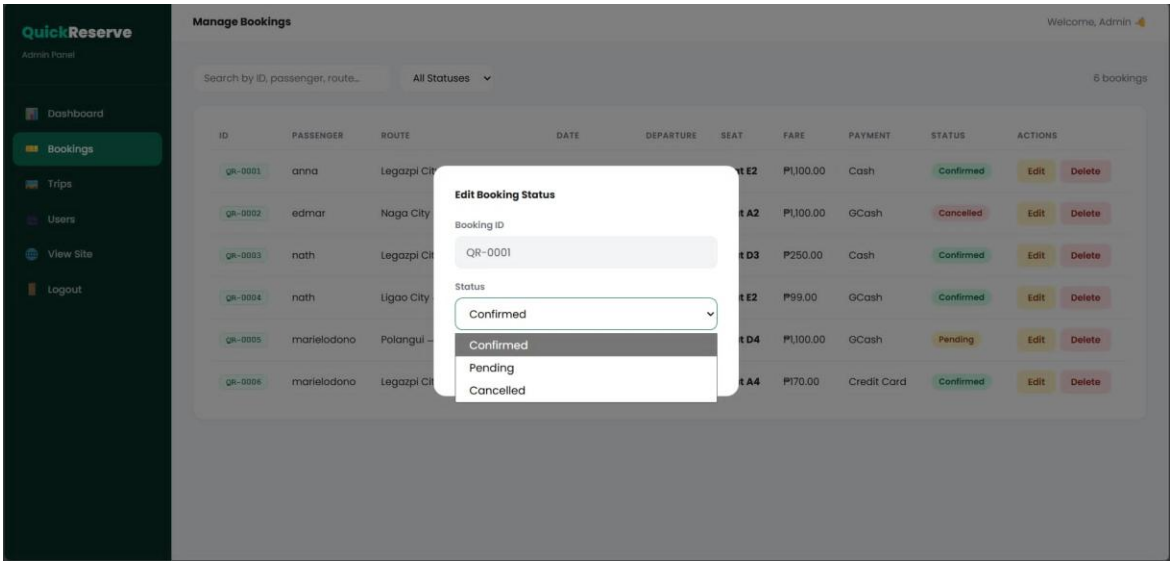
➢ Delete Booking Confirmation Page

This screenshot shows the **Delete Booking Confirmation** dialog in the admin panel. Before removing a booking record, the system asks administrators to confirm the action to prevent accidental deletion. The dialog provides options to either proceed with or cancel the deletion process.



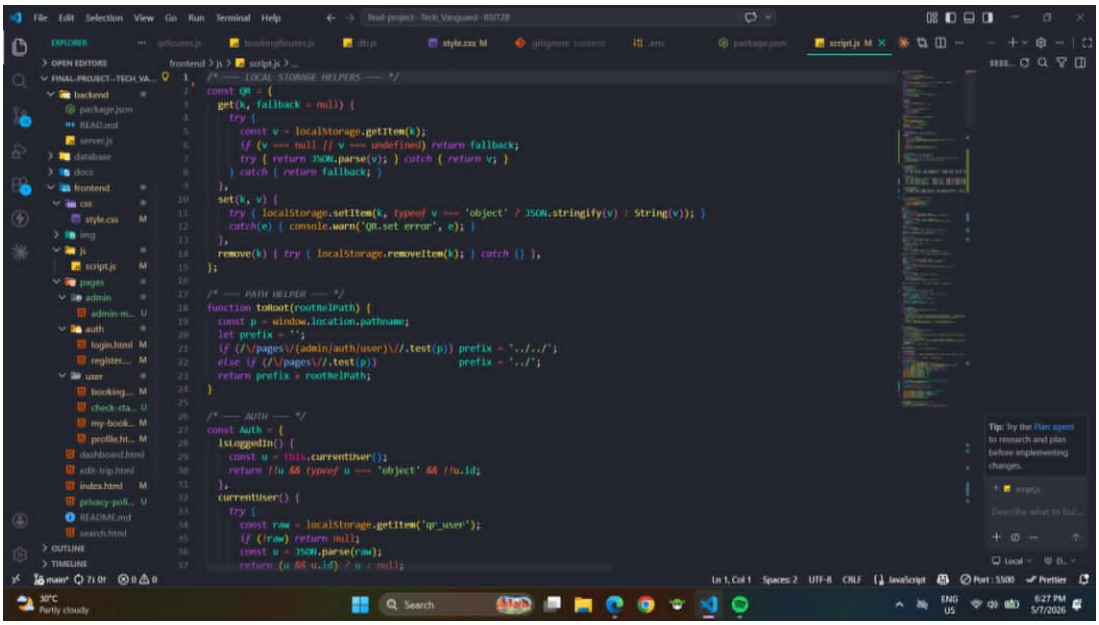
➢ Edit Booking Status Page

This screenshot displays the **Edit Booking Status** feature in the admin panel. Administrators can update the booking status of passengers by selecting options such as **Confirmed, Pending,** or **Cancelled**. This function helps admins manage and monitor reservation transactions efficiently.

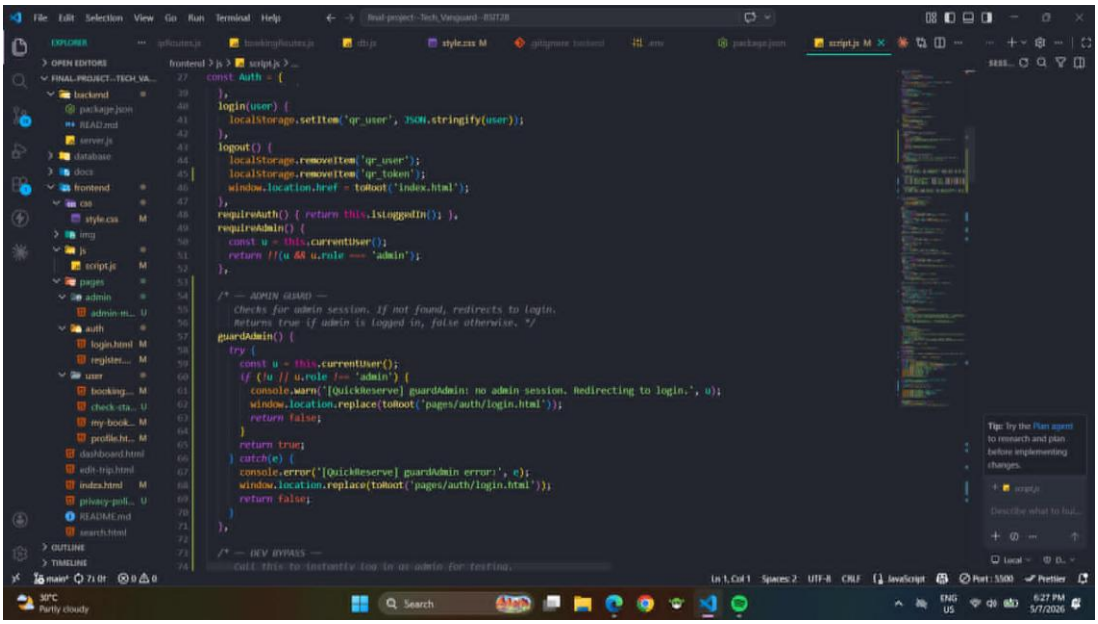


IV. JavaScript Snippets

QuickReserve system for handling local storage management, authentication, admin access control, and default data initialization. The code demonstrates functions for storing and retrieving user data, managing login sessions, validating admin access, redirecting unauthorized users, and seeding sample users, trips, and booking records to support the system's reservation and management functionalities during development and testing.



```
/* ── LOCAL STORAGE HELPERS ── */
const QR = {
  get(k, fallback = null) {
    try {
      const v = localStorage.getItem(k);
      if (v == null || v === undefined) return fallback;
      try { return JSON.parse(v); } catch { return v; }
    } catch { return fallback; }
  },
  set(k, v) {
    try { localStorage.setItem(k, typeof v === 'object' ? JSON.stringify(v) : String(v)); }
    catch(e) { console.warn('QR.set error', e); }
  },
  remove(k) { try { localStorage.removeItem(k); } catch {} },
};

/* ── PATH HELPER ── */
function toRoot(rootRelPath) {
  const p = window.location.pathname;
  let prefix = '';
  if (/\/pages\/(admin|auth|user)\//.test(p)) prefix = '../../';
  else if (/\/pages\//.test(p))               prefix = '../';
  return prefix + rootRelPath;
}

/* ── AUTH ── */
const Auth = {
  isLoggedIn() {
    const u = this.currentUser();
    return !!u && typeof u === 'object' && !!u.id;
  },
  currentUser() {
    try {
      const raw = localStorage.getItem('qr_user');
      if (!raw) return null;
      const u = JSON.parse(raw);
      return (u && u.id) ? u : null;
```



```
  },
  login(user) {
    localStorage.setItem('qr_user', JSON.stringify(user));
  },
  logout() {
    localStorage.removeItem('qr_user');
    localStorage.removeItem('qr_token');
    window.location.href = toRoot('index.html');
  },
  requireAuth() { return this.isLoggedIn(); },
  requireAdmin() {
    const u = this.currentUser();
    return !!(u && u.role === 'admin');
  },

  /* ── ADMIN GUARD ──
     Checks for admin session. If not found, redirects to login.
     Returns true if admin is logged in, false otherwise. */
  guardAdmin() {
    try {
      const u = this.currentUser();
      if (!u || u.role !== 'admin') {
        console.warn('[QuickReserve] guardAdmin: no admin session. Redirecting to login.', u);
        window.location.replace(toRoot('pages/auth/login.html'));
        return false;
      }
      return true;
    } catch(e) {
      console.error('[QuickReserve] guardAdmin error:', e);
      window.location.replace(toRoot('pages/auth/login.html'));
      return false;
    }
  },

  /* ── DEV BYPASS ──
     Call this to instantly log in as admin for testing.
```

```
const Auth = {
  /* — DEV BYPASS —
     Call this to instantly log in as admin for testing.
     Open browser console and type: Auth.devLogin() */
  devLogin() {
    const adminUser = { id: 1, name: 'Admin User', email: 'admin@quickreserve.com', role: 'admin' };
    this.login(adminUser);
    console.log('[QuickReserve] Dev admin login successful. Reloading page...');
    window.location.reload();
  },
};

/* ═══ SEED DEFAULT DATA ═══ */
function seedData() {
  if (!QR.get('qr_users')) {
    QR.set('qr_users', [
      { id: 1, name: 'Admin User',     email: 'admin@quickreserve.com', password: 'admin123', role: 'admin', phone: '090012345
      { id: 2, name: 'Maria Santos',   email: 'maria@example.com',      password: 'user123',  role: 'user',  phone: '091712345
      { id: 3, name: 'Juan dela Cruz', email: 'juan@example.com',       password: 'user123',  role: 'user',  phone: '092812345
    ]);
  }
  if (!QR.get('qr_trips')) {
    QR.set('qr_trips', [
      { id: 1, route: 'Legazpi City → Sorsogon City', duration: '1-2 hrs',    priceMin: 150, priceMax: 250, departures: ['06:0
      { id: 2, route: 'Legazpi City → Naga City',     duration: '2-3 hrs',    priceMin: 200, priceMax: 350, departures: ['06:0
      { id: 3, route: 'Naga City → Daet',             duration: '3-4 hrs',    priceMin: 300, priceMax: 450, departures: ['07:0
      { id: 4, route: 'Iriga City → Naga City',       duration: '1-2 hrs',    priceMin: 150, priceMax: 250, departures: ['06:0
      { id: 5, route: 'Ligao City → Legazpi City',    duration: '30-45 mins', priceMin: 80,  priceMax: 150, departures: ['06:0
      { id: 6, route: 'Polangui → Legazpi City',      duration: '45-60 mins', priceMin: 60,  priceMax: 120, departures: ['06:0
    ]);
  }
  if (!QR.get('qr_bookings')) {
    QR.set('qr_bookings', [
      { id: 'QR-0001', userId: 2, userName: 'Maria Santos',   route: 'Legazpi City → Naga City',     seat: 'A1', date: '2026-0
      { id: 'QR-0002', userId: 3, userName: 'Juan dela Cruz', route: 'Legazpi City → Sorsogon City', seat: 'B1', date: '2026-0
      { id: 'QR-0003', userId: 2, userName: 'Maria Santos',   route: 'Naga City → Daet',             seat: 'C2', date: '2026-0
    ]);
```

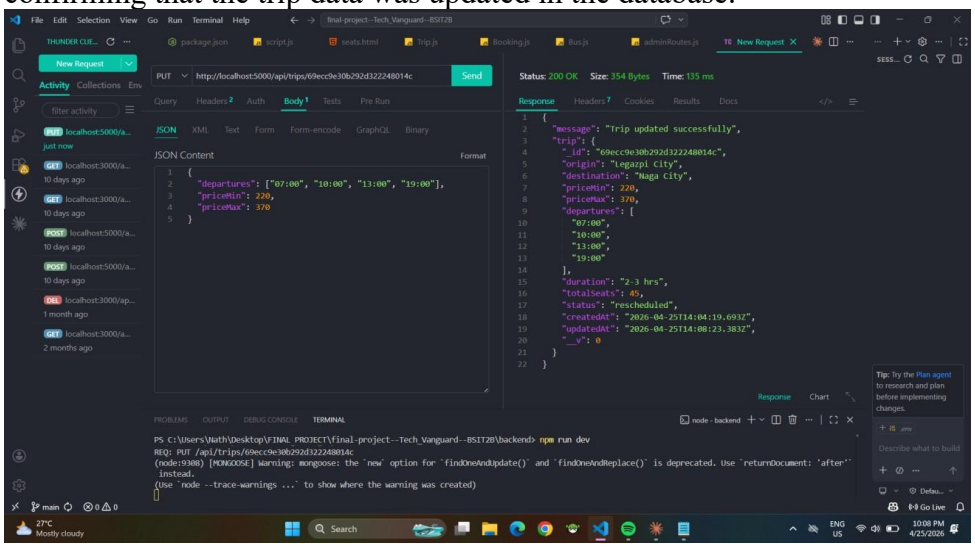
```
// Clear stale data and re-seed if routes are missing
(function migrateRouteData() {
  try {
    var trips = localStorage.getItem('qr_trips');
    if (trips && trips.indexOf('Legazpi') === -1) {
      localStorage.removeItem('qr_trips');
      localStorage.removeItem('qr_bookings');
    }
  } catch(e) {}
})();
seedData();

/* ═══ BOOKINGS CRUD ═══ */
const Bookings = {
  all()       { return QR.get('qr_bookings', []); },
  forUser(uid) { return this.all().filter(b => Number(b.userId) === Number(uid)); },
  byId(id)    { return this.all().find(b => b.id === id); },
  add(booking) {
    const all = this.all();
    const id  = 'QR-' + String(all.length + 1).padStart(4, '0');
    const nb  = { id, ...booking, createdAt: new Date().toISOString().split('T')[0] };
    all.push(nb);
    QR.set('qr_bookings', all);
    return nb;
  },
  update(id, data) {
    const all = this.all();
    const i   = all.findIndex(b => b.id === id);
    if (i > -1) { all[i] = { ...all[i], ...data }; QR.set('qr_bookings', all); }
  },
  cancel(id) { this.update(id, { status: 'cancelled' }); },
  delete(id) { QR.set('qr_bookings', this.all().filter(b => b.id !== id)); },
};

/* ═══ TRIPS CRUD ═══ */
const Trips = {
```

V. Backend Routes and Controller Snippets

PUT ROUTE

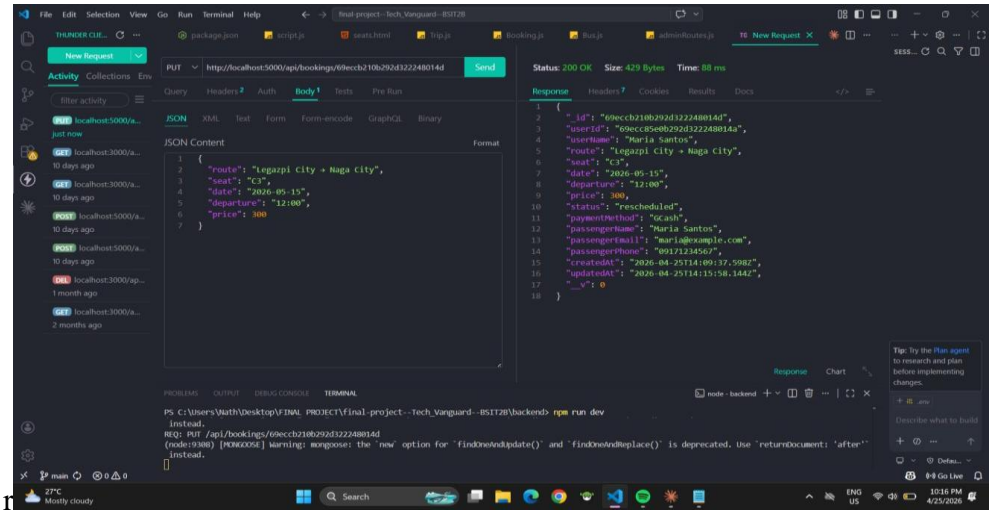
Update Trip / PUT request to update trip details.

The system uses a PUT request to update trip information such as origin, destination, departure time, and pricing. The request was tested using Thunder Client and returned a successful response, confirming that the trip data was updated in the database.



Reschedule Booking / PUT request to update booking details

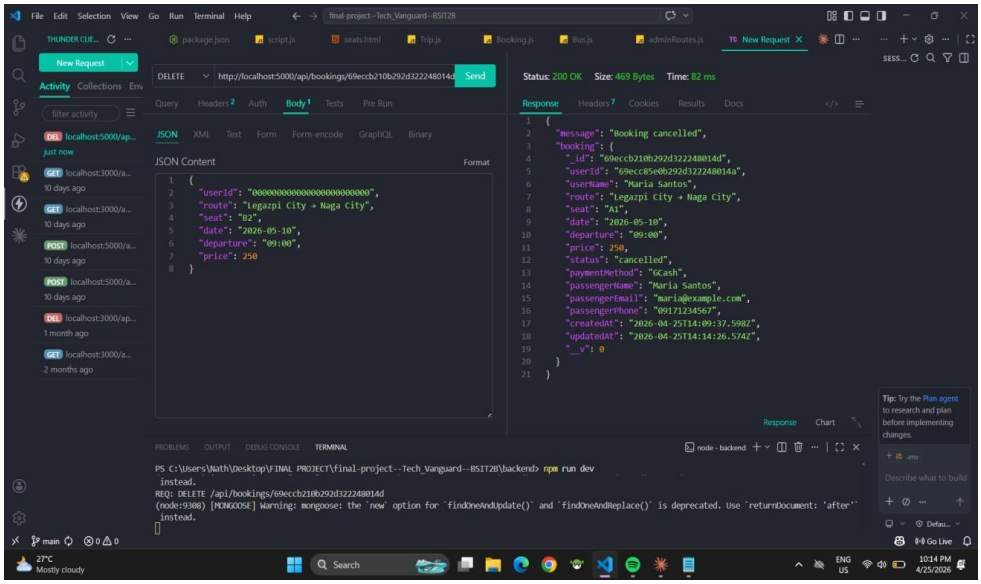
The PUT request updated the booking details and changed its status to **rescheduled**. The 200 OK response confirms that the update was successful in the database.



DELETE ROUTE

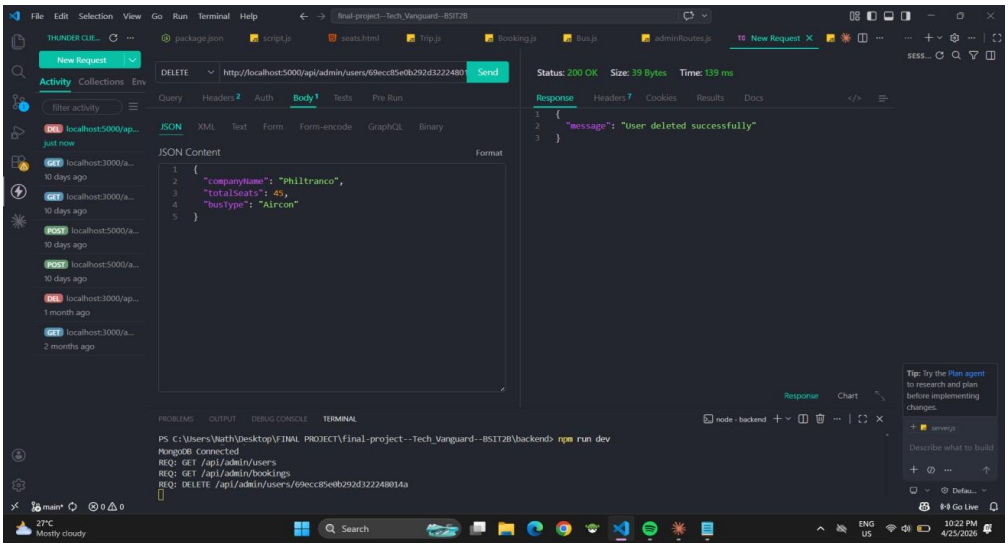
Cancel Booking / DELETE request to remove a booking.

The DELETE request was used to remove a booking record in the system. Instead of permanently deleting the data, the system updates the booking status to **"cancelled"**, allowing the record to remain stored for tracking purposes. This confirms that the deletion process was successfully handled by the system.



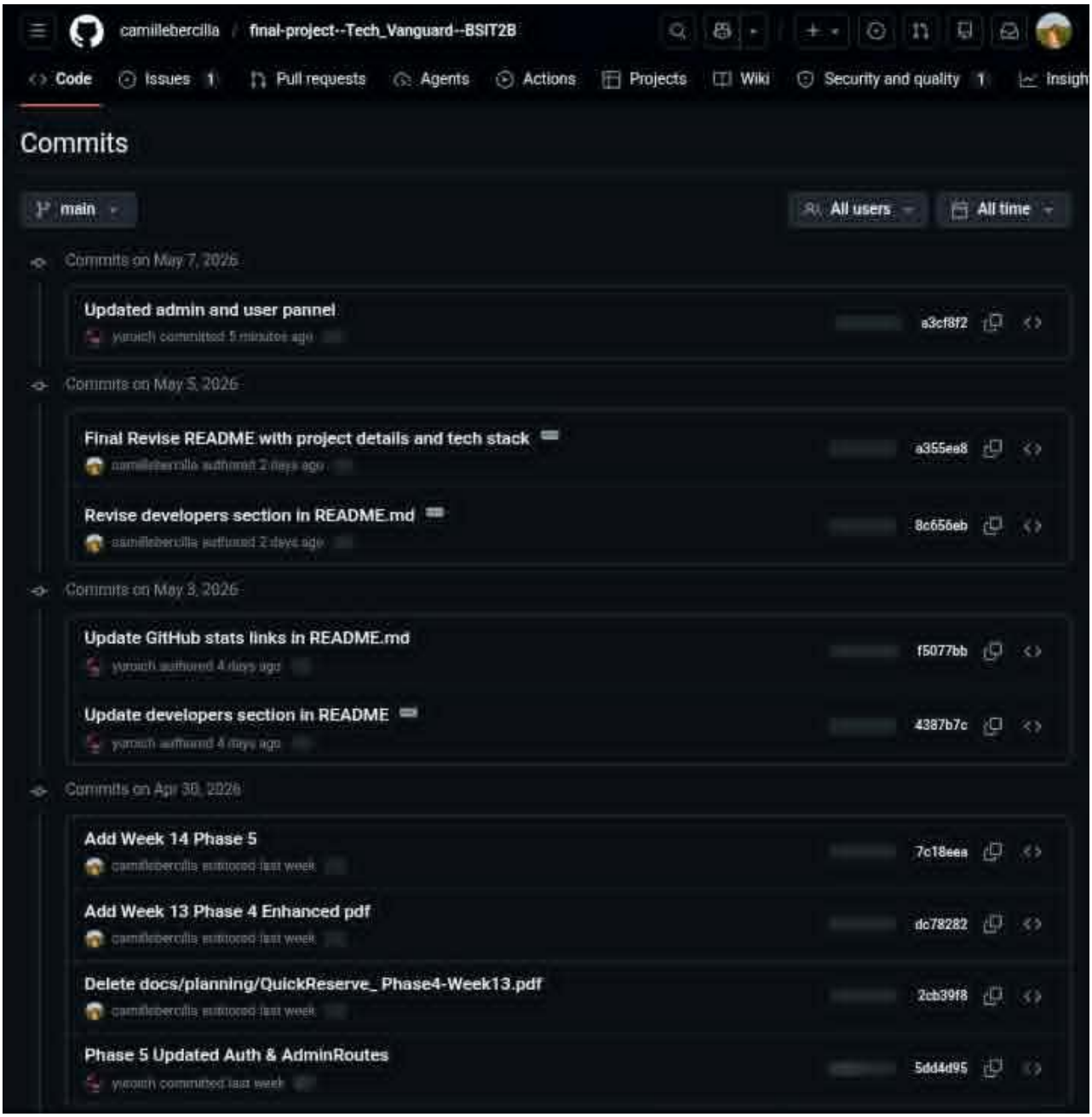
Admin — Delete User / DELETE request to remove a user

The DELETE request allows the administrator to remove a user record from the system. This action deletes the selected user from the database, ensuring that unnecessary or inactive accounts are properly managed.



VI. GitHub Commit Logs

The following commits show the implementation of update and delete functionalities in both frontend and backend.



VII . Individual reflections from each group member (3–5 sentences)

Each member of the group was given a specific responsibility to contribute to the successful development, testing, and documentation of the **Quick Reserve:Online Bus & Travel Booking System.**

Nathaniel O. Raynada – Project Manager / Front-end Developer

As the group leader and frontend developer, I was responsible for implementing the edit and delete features in the user interface, including buttons, modals, and event handling. I ensured that these frontend elements were properly connected to the backend PUT and DELETE endpoints using JavaScript and fetch requests. I also guided the team in coordinating tasks and making sure our work was aligned and functional as a whole. This experience improved both my frontend development skills and my ability to lead and collaborate effectively.

Anna Mae P. Broma – Back-End Developer

As a backend developer in Phase 6 , I implemented and tested the PUT and DELETE API endpoints to ensure data updates and deletions work correctly in the system. I made sure that all changes from the frontend are properly reflected in the database and that the endpoints respond accurately. This phase improved my understanding of completing full CRUD operations and maintaining reliable backend functionality.

Camille Bercilla – GitHub Manager

I made it possible to organize the process of pushing documentation updates. All changes were pushed to the main or phase-6 branch, with working deployment or local testing results included. I also updated the GitHub repository by committing all new files to keep everything aligned.

Mariel O. Odoño – Documentation Officer

I prepared the documentation by organizing all screenshots and code snippets. I ensured that all required sections were complete and clearly explained. This improved my skills in technical documentation.

John Edmar Bongcodin – System Tester

I worked as a tester and debugger by checking if the update and delete features were working properly. I tested things like edit forms, confirmation messages, and made sure the changes were saved correctly in the database. This phase helped me get better at finding errors and understanding how CRUD operations work in a real system.

Justin Dasallas – Database Manager

I verified that all updates and deletions are reflected correctly in the MongoDB database. I checked that no incorrect or duplicate data remained after operations. This ensured data consistency and reliability.